Information Retrieval in Healthcare

Practical Assignment | B.Tech AI-ML | IGDTUW | April 2026

Student: Nishika | **Roll No:** 04001192022

Models Implemented: TF-IDF | BM25 | Dense Semantic Retrieval (sentence-transformers + FAISS)

Dataset: MedQuAD — Medical Question Answering Dataset (Kaggle / NIH)

Metrics: Precision@K | Recall@K | MAP | MRR

```
# Install all required libraries
!pip install -q rank_bm25 sentence-transformers faiss-cpu kaggle

# Download required NLTK data
import nltk
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)

print("All dependencies installed.")
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 23.8/23.8 MB 34.6 MB/s eta 0:00:00
All dependencies installed.
```

```
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

# NLP
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# Classical IR
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from rank_bm25 import BM25Okapi

# Dense IR
from sentence_transformers import SentenceTransformer
import faiss

print("All libraries imported successfully.")
```

```
All libraries imported successfully.
```

Cell 3: Load MedQuAD Dataset from Kaggle

> **Before running:** Go to kaggle.com → Profile → Settings → API → Create New Token → download `kaggle.json`

```
# Step 1: Upload your Dataset
from google.colab import files
files.upload()
```

```
import pandas as pd

df = pd.read_csv('/content/medquad.csv')

print("Dataset shape:", df.shape)
print("\nColumn names:", df.columns.tolist())
print("\nFirst few rows:")
df.head(3)
```

```
Dataset shape: (16412, 4)

Column names: ['question', 'answer', 'source', 'focus_area']

First few rows:
```

	question	answer	source	focus_area
0	What is (are) Glaucoma ?	Glaucoma is a group of diseases that can damag...	NIHSeniorHealth	Glaucoma
1	What causes Glaucoma ?	Nearly 2.7 million people have glaucoma, a lea...	NIHSeniorHealth	Glaucoma
2	What are the symptoms of Glaucoma ?	Symptoms of Glaucoma Glaucoma can develop in ...	NIHSeniorHealth	Glaucoma

```
# Step 3: Clean and prepare the corpus
# Drop rows where question or answer is missing
df = df.dropna(subset=['question', 'answer'])
df = df[df['answer'].str.strip() != '']
df = df[df['question'].str.strip() != '']

# Use first 1000 rows for faster processing in Colab (remove limit for full run)
df = df.head(1000).reset_index(drop=True)

# Extract questions and answers
corpus    = df['answer'].tolist()    # These are the documents to search through
questions = df['question'].tolist()  # These are used as queries

print(f"Corpus size: {len(corpus)} medical documents")
print(f"\nSample question: {questions[0]}")
print(f"\nSample answer (first 200 chars): {corpus[0][:200]}...")
```

```
Corpus size: 1000 medical documents

Sample question: What is (are) Glaucoma ?

Sample answer (first 200 chars): Glaucoma is a group of diseases that can damage the eye's optic nerve and result
```

Cell 4: Text Preprocessing

```
# Initialize NLP tools
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    """
    Preprocess text for classical IR models (TF-IDF and BM25).
    Steps:
      1. Lowercase
      2. Remove special characters and digits
      3. Tokenize into words
      4. Remove stopwords (the, is, in, etc.)
      5. Lemmatize (running -> run, diseases -> disease)
    Note: Dense retrieval uses raw text — no preprocessing needed.
    """
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)       # Keep only letters and spaces
    tokens = word_tokenize(text)                 # Split into words
    tokens = [
        lemmatizer.lemmatize(t)
        for t in tokens
        if t not in stop_words and len(t) > 2   # Remove stopwords and very short words
    ]
    return ' '.join(tokens)

# Preprocess all documents and questions
print("Preprocessing corpus...")
processed_corpus    = [preprocess(doc) for doc in corpus]
processed_questions = [preprocess(q)   for q   in questions]

print("Done.")
print(f"\nOriginal : {corpus[0][:100]}")
print(f"\nProcessed: {processed_corpus[0][:100]}")
```

```
Preprocessing corpus...
Done.

Original : Glaucoma is a group of diseases that can damage the eye's optic nerve and result in vision loss and

Processed: glaucoma group disease damage eye optic nerve result vision loss blindness glaucoma strike anyone ri
```

Cell 5: TF-IDF Retrieval Model

```
# ----------------------------------------------------------------
# TF-IDF (Term Frequency - Inverse Document Frequency)
#
# TF  = how often a word appears in a document
# IDF = penalizes words that appear in many documents (less unique)
# Score = TF * IDF
#
# Retrieval: cosine similarity between query vector and doc vectors
# ----------------------------------------------------------------

tfidf_vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),   # Use single words and word pairs (bigrams)
    max_df=0.95,          # Ignore words appearing in >95% of documents
    min_df=1,             # Must appear in at least 1 document
    sublinear_tf=True     # Use log(TF) to reduce effect of very frequent terms
)

# Build document-term matrix (each row = a document, each column = a term)
tfidf_matrix = tfidf_vectorizer.fit_transform(processed_corpus)

print(f"TF-IDF matrix: {tfidf_matrix.shape[0]} documents x {tfidf_matrix.shape[1]} terms")

def tfidf_retrieve(query, top_k=5):
    """
    Retrieve top-K relevant documents for a given query using TF-IDF.
    """
    query_vec = tfidf_vectorizer.transform([preprocess(query)])  # Vectorize query
    scores    = cosine_similarity(query_vec, tfidf_matrix).flatten()  # Compare with all docs
    top_idx   = np.argsort(scores)[::-1][:top_k]  # Sort descending, take top-K
    return top_idx, scores[top_idx]

# Quick test
test_query = "What are the symptoms of diabetes?"
idx, scores = tfidf_retrieve(test_query, top_k=3)
print(f"\nTF-IDF Results for: '{test_query}'")
for rank, (i, s) in enumerate(zip(idx, scores), 1):
    print(f"  [{rank}] Score={s:.4f} | {corpus[i][:120]}...")
```

```
TF-IDF matrix: 1000 documents x 66075 terms

TF-IDF Results for: 'What are the symptoms of diabetes?'
  [1] Score=0.1062 | Diabetes is often called a "silent" disease because it can cause serious complications even
  [2] Score=0.0691 | Many people with diabetes experience one or more symptoms, including extreme thirst or hunge
  [3] Score=0.0677 | Diabetes is a very serious disease. Over time, diabetes that is not well managed causes seri
```

Cell 6: BM25 Retrieval Model

```
# ----------------------------------------------------------------
# BM25 (Best Match 25) — Probabilistic IR Model
#
# Improvements over TF-IDF:
#   - Term saturation: a word repeated 100x doesn't score 100x more
#   - Length normalization: long docs not unfairly preferred
#
# Parameters:
#   k1 = 1.5  (controls term frequency saturation)
#   b  = 0.75 (controls document length normalization)
# ----------------------------------------------------------------

# BM25 requires tokenized corpus (list of token lists)
tokenized_corpus = [doc.split() for doc in processed_corpus]

# Build BM25 index
bm25 = BM25Okapi(tokenized_corpus, k1=1.5, b=0.75)

print(f"BM25 index ready. Average document length: {bm25.avgdl:.1f} tokens")

def bm25_retrieve(query, top_k=5):
    """
    Retrieve top-K relevant documents using BM25 scoring.
    """
    query_tokens = preprocess(query).split()          # Preprocess and tokenize query
    scores       = bm25.get_scores(query_tokens)       # Score all documents
    top_idx      = np.argsort(scores)[::-1][:top_k]   # Top-K descending
    return top_idx, scores[top_idx]
```

```
# Quick test
test_query = "What are the symptoms of diabetes?"
idx, scores = bm25_retrieve(test_query, top_k=3)
print(f"\nBM25 Results for: '{test_query}'")
for rank, (i, s) in enumerate(zip(idx, scores), 1):
    print(f"  [{rank}] Score={s:.4f} | {corpus[i][:120]}...")
```

```
BM25 index ready. Average document length: 182.2 tokens

BM25 Results for: 'What are the symptoms of diabetes?'
  [1] Score=6.4758 | Diabetes is often called a "silent" disease because it can cause serious complications even
  [2] Score=5.9618 | Common Symptoms Some people with P.A.D. do not have any symptoms. Others may have a number o
  [3] Score=5.9147 | Too Much Glucose in the Blood Diabetes means your blood glucose (often called blood sugar) i
```

Cell 7: Dense Semantic Retrieval (sentence-transformers + FAISS)

```
# ----------------------------------------------------------------
# Dense Semantic Retrieval
#
# Instead of matching keywords, this approach:
#   1. Encodes every document as a 384-dimensional vector
#      (numbers that represent the *meaning* of the text)
#   2. At query time, encodes the query the same way
#   3. Finds documents whose vectors are closest to the query vector
#
# Model: all-MiniLM-L6-v2 (~22MB, fast, good quality)
# Index: FAISS (Facebook AI Similarity Search) — efficient nearest neighbor
# ----------------------------------------------------------------

print("Loading sentence-transformer model...")
sbert = SentenceTransformer('all-MiniLM-L6-v2')

print("Encoding documents (this takes ~1-2 min for 1000 docs on CPU)...")
# Encode all documents into dense vectors
doc_embeddings = sbert.encode(
    corpus,
    batch_size=64,
    show_progress_bar=True,
    convert_to_numpy=True
)

# Normalize for cosine similarity (inner product after L2 normalization = cosine)
faiss.normalize_L2(doc_embeddings)

# Build FAISS index
dim   = doc_embeddings.shape[1]      # 384 dimensions
index = faiss.IndexFlatIP(dim)       # Flat index, inner product
index.add(doc_embeddings)

print(f"\nFAISS index ready: {index.ntotal} vectors, {dim} dimensions")

def dense_retrieve(query, top_k=5):
    """
    Retrieve top-K relevant documents using dense semantic similarity.
    Uses raw query text — no preprocessing needed.
    """
    q_emb = sbert.encode([query], convert_to_numpy=True)  # Encode query
    faiss.normalize_L2(q_emb)                              # Normalize
    scores, indices = index.search(q_emb, top_k)           # Search FAISS
    return indices[0], scores[0]

# Quick test
test_query = "What are the symptoms of diabetes?"
idx, scores = dense_retrieve(test_query, top_k=3)
print(f"\nDense Retrieval Results for: '{test_query}'")
for rank, (i, s) in enumerate(zip(idx, scores), 1):
    print(f"  [{rank}] Score={s:.4f} | {corpus[i][:120]}...")
```

```
Loading sentence-transformer model...
modules.json: 100%                                                    349/349 [00:00<00:00, 27.8kB/s]
config_sentence_transformers.json: 100%                               116/116 [00:00<00:00, 9.75kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limi
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
README.md:       10.5k/? [00:00<00:00, 859kB/s]
sentence_bert_config.json: 100%                                       53.0/53.0 [00:00<00:00, 5.39kB/s]
config.json: 100%                                                     612/612 [00:00<00:00, 59.3kB/s]
model.safetensors: 100%                                               90.9M/90.9M [00:02<00:00, 210MB/s]
Loading weights: 100%                                                 103/103 [00:00<00:00, 412.75it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED   :can be ignored when loading from different task/architecture; not ok if you expect identical arch
tokenizer_config.json: 100%                                           350/350 [00:00<00:00, 20.1kB/s]
vocab.txt:       232k/? [00:00<00:00, 12.0MB/s]
tokenizer.json:       466k/? [00:00<00:00, 19.3MB/s]
special_tokens_map.json: 100%                                         112/112 [00:00<00:00, 10.9kB/s]
config.json: 100%                                                     190/190 [00:00<00:00, 17.5kB/s]
Encoding documents (this takes ~1-2 min for 1000 docs on CPU)...
Batches: 100%                                                         16/16 [02:13<00:00,  3.74s/it]

FAISS index ready: 1000 vectors, 384 dimensions

Dense Retrieval Results for: 'What are the symptoms of diabetes?'
  [1] Score=0.7694 | Many people with diabetes experience one or more symptoms, including extreme thirst or hunge
  [2] Score=0.7539 | Diabetes is often called a "silent" disease because it can cause serious complications even
  [3] Score=0.5349 | Diabetes means your blood glucose (often called blood sugar) is too high. Your blood always
```

Cell 8: Evaluation — Precision, Recall, MAP, MRR

```
# ----------------------------------------------------------------
# Evaluation Setup
#
# Ground truth: for each question at index i, the correct
# document is the answer at the same index i in the corpus.
# So ground_truth[i] = [i] (one relevant document per query).
# ----------------------------------------------------------------

ground_truth = {i: [i] for i in range(len(questions))}

def precision_at_k(retrieved, relevant, k):
    """Precision@K: what fraction of top-K results are relevant?"""
    return sum(1 for r in retrieved[:k] if r in relevant) / k

def recall_at_k(retrieved, relevant, k):
    """Recall@K: what fraction of all relevant docs did we find in top-K?"""
    return sum(1 for r in retrieved[:k] if r in relevant) / len(relevant)

def average_precision(retrieved, relevant):
    """AP: precision averaged at each position where a relevant doc appears."""
    hits, score = 0, 0.0
    for i, r in enumerate(retrieved, 1):
        if r in relevant:
            hits  += 1
            score += hits / i
    return score / len(relevant) if relevant else 0.0

def reciprocal_rank(retrieved, relevant):
    """RR: 1 / rank of the first relevant result."""
    for i, r in enumerate(retrieved, 1):
        if r in relevant:
            return 1.0 / i
    return 0.0

def evaluate(retrieve_fn, queries, ground_truth, k=5):
    """
    Evaluate a retrieval function over all queries.
```

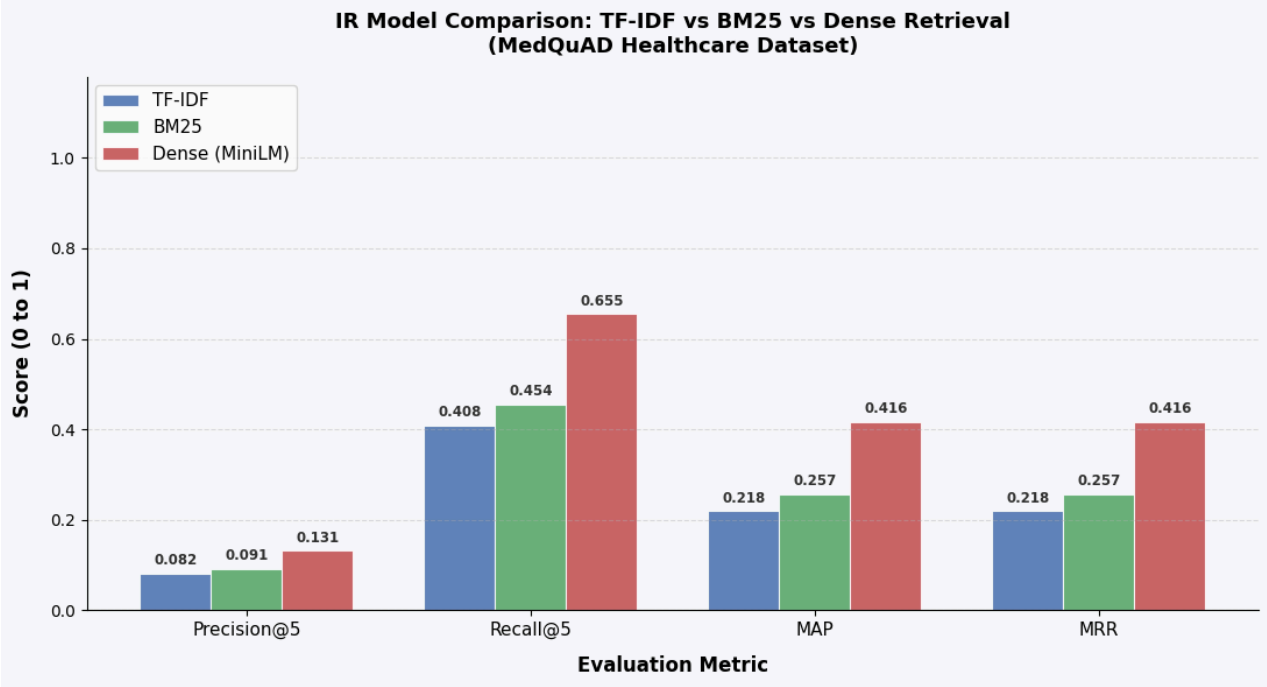
```
    Returns dict of averaged metrics.
    """
    p_scores, r_scores, ap_scores, rr_scores = [], [], [], []
    for i, query in enumerate(queries):
        relevant  = set(ground_truth[i])
        retrieved = list(retrieve_fn(query, top_k=k)[0])
        p_scores.append(precision_at_k(retrieved, relevant, k))
        r_scores.append(recall_at_k(retrieved, relevant, k))
        ap_scores.append(average_precision(retrieved, relevant))
        rr_scores.append(reciprocal_rank(retrieved, relevant))
    return {
        f'Precision@{k}': round(np.mean(p_scores), 4),
        f'Recall@{k}'   : round(np.mean(r_scores), 4),
        'MAP'           : round(np.mean(ap_scores), 4),
        'MRR'           : round(np.mean(rr_scores), 4),
    }

# Run evaluation for all three models
K = 5
print("Evaluating TF-IDF...")
tfidf_scores = evaluate(tfidf_retrieve, questions, ground_truth, k=K)

print("Evaluating BM25...")
bm25_scores  = evaluate(bm25_retrieve,  questions, ground_truth, k=K)

print("Evaluating Dense Retrieval...")
dense_scores = evaluate(dense_retrieve, questions, ground_truth, k=K)

# Display as table
results_df = pd.DataFrame({
    'TF-IDF'        : tfidf_scores,
    'BM25'          : bm25_scores,
    'Dense (MiniLM)': dense_scores,
}).T

print("\n========== EVALUATION RESULTS ==========")
print(results_df.to_string())
```

```
Evaluating TF-IDF...
Evaluating BM25...
Evaluating Dense Retrieval...

========== EVALUATION RESULTS ==========
                 Precision@5  Recall@5     MAP     MRR
TF-IDF               0.0816     0.408  0.2185  0.2185
BM25                 0.0908     0.454  0.2569  0.2569
Dense (MiniLM)       0.1310     0.655  0.4157  0.4157
```

Cell 9: Visualization — Bar Chart Comparison

```
# Bar chart comparing all three models across all four metrics

metrics = list(tfidf_scores.keys())
models  = ['TF-IDF', 'BM25', 'Dense (MiniLM)']
colors  = ['#4C72B0', '#55A868', '#C44E52']
all_results = [tfidf_scores, bm25_scores, dense_scores]

x     = np.arange(len(metrics))
width = 0.25

fig, ax = plt.subplots(figsize=(11, 6))
fig.patch.set_facecolor('#F8F9FA')
ax.set_facecolor('#F8F9FA')

for i, (model, color, result) in enumerate(zip(models, colors, all_results)):
    vals = [result[m] for m in metrics]
    bars = ax.bar(x + (i - 1) * width, vals, width,
                  label=model, color=color, alpha=0.87,
                  edgecolor='white', linewidth=0.8)
    # Add value labels on bars
    for bar, val in zip(bars, vals):
        ax.text(bar.get_x() + bar.get_width() / 2,
                bar.get_height() + 0.012,
                f'{val:.3f}', ha='center', va='bottom',
                fontsize=8.5, fontweight='bold', color='#333333')

ax.set_xlabel('Evaluation Metric', fontsize=12, fontweight='bold', labelpad=10)
ax.set_ylabel('Score (0 to 1)', fontsize=12, fontweight='bold', labelpad=10)
ax.set_title('IR Model Comparison: TF-IDF vs BM25 vs Dense Retrieval\n(MedQuAD Healthcare Dataset)',
             fontsize=13, fontweight='bold', pad=15)
ax.set_xticks(x)
```

```
ax.set_xticklabels(metrics, fontsize=11)
ax.set_ylim(0, 1.18)
ax.legend(fontsize=11, loc='upper left', framealpha=0.8)
ax.grid(axis='y', linestyle='--', alpha=0.4)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.savefig('ir_comparison_chart.png', dpi=150, bbox_inches='tight')
plt.show()
print("Chart saved as ir_comparison_chart.png")
print("(Take a screenshot of this chart for your Word document)")
```




```
Chart saved as ir_comparison_chart.png
(Take a screenshot of this chart for your Word document)
```

Cell 10: Interactive Demo — Compare All Models on Any Query

```
def compare_all(query, top_k=3):
    """
    Run all three models on a query and show results side by side.
    This is the core function of the IR system — enter any medical
    question and see how each model responds.
    """
    print(f"\n{'='*70}")
    print(f"  QUERY: {query}")
    print(f"{'='*70}")

    for model_name, fn in [('TF-IDF', tfidf_retrieve),
                           ('BM25',   bm25_retrieve),
                           ('Dense',  dense_retrieve)]:
        print(f"\n--- {model_name} ---")
        idx_list, score_list = fn(query, top_k=top_k)
        for rank, (idx, score) in enumerate(zip(idx_list, score_list), 1):
            preview = corpus[int(idx)][:130].replace('\n', ' ')
            print(f"  [{rank}] (score={score:.4f})  {preview}...")

# Test with queries that show semantic gap between models
queries = [
    "How is high blood sugar managed?",          # Semantically = diabetes
    "What happens during a cardiac event?",       # Semantically = heart attack
    "treatment for bacterial lung infection",     # Semantically = pneumonia
    "memory loss in elderly patients",            # Semantically = Alzheimer's
]

for q in queries:
    compare_all(q, top_k=3)
```

```
======================================================================
  QUERY: How is high blood sugar managed?
======================================================================

--- TF-IDF ---
  [1] (score=0.1658)  You can lower your risk of having a heart attack, even if you have already had a heart at
  [2] (score=0.1594)  Too much cholesterol in your blood is called high blood cholesterol. It can be serious. P
  [3] (score=0.1349)  High blood pressure is a common disease in which blood flows through blood vessels (arter

--- BM25 ---
  [1] (score=10.9151)  Too Much Glucose in the Blood Diabetes means your blood glucose (often called blood suga
  [2] (score=10.7253)  You can lower your risk of having a heart attack, even if you have already had a heart a
  [3] (score=10.0768)  "DASH" stands for "Dietary Approaches to Stop Hypertension." This is the name of a clini

--- Dense ---
  [1] (score=0.5817)  Diabetes cannot be cured, but it can be managed. Managing blood glucose (blood sugar) as
  [2] (score=0.5682)  Diabetes means your blood glucose (often called blood sugar) is too high. Your blood alwa
  [3] (score=0.5677)  People with type 1 diabetes control their blood sugar with insulin -- either with shots o

======================================================================
  QUERY: What happens during a cardiac event?
======================================================================

--- TF-IDF ---
  [1] (score=0.1040)  Several tests are used to diagnose a heart attack. - An electrocardiogram, also called an
  [2] (score=0.0890)  Mild cognitive impairment, or MCI, is a condition that can be an early sign of Alzheimers
  [3] (score=0.0828)  If You Have Symptoms, Call 9-1-1 Diagnosis and treatment of a heart attack can begin when

--- BM25 ---
  [1] (score=11.0609)  Several tests are used to diagnose a heart attack. - An electrocardiogram, also called a
  [2] (score=9.3494)  If You Have Symptoms, Call 9-1-1 Diagnosis and treatment of a heart attack can begin when
  [3] (score=8.8399)  The risk factors for depression are family history, life experiences, and environment. If

--- Dense ---
  [1] (score=0.5816)  A heart attack occurs when the supply of blood and oxygen to an area of the heart muscle
  [2] (score=0.4732)  The coronary arteries bring blood to your heart. If plaque builds up in these arteries, t
  [3] (score=0.4679)  Blood Flow to the Heart Is Blocked The heart works 24 hours a day, pumping oxygen and nut

======================================================================
  QUERY: treatment for bacterial lung infection
======================================================================

--- TF-IDF ---
  [1] (score=0.1655)  COPD is a disease that slowly worsens over time, especially if you continue to smoke. If
  [2] (score=0.1242)  Call your doctor right away if your symptoms worsen suddenly. People with COPD may have s
  [3] (score=0.0831)  Some balance disorders are caused by problems in the inner ear. The part of the inner ear

--- BM25 ---
  [1] (score=10.7067)  Some balance disorders are caused by problems in the inner ear. The part of the inner ea
  [2] (score=10.4911)  The most common complication of shingles is pain -- a condition called post-herpetic neu
  [3] (score=9.9628)  When gingivitis is not treated, it can advance to periodontitis (which means "inflammatio

--- Dense ---
  [1] (score=0.4653)  If you have COPD, you can take these steps to prevent complications and control the disab
  [2] (score=0.4494)  Small cell lung cancer spreads quickly. In many cases, cancer cells have already spread t
  [3] (score=0.4447)  If you smoke, the most important thing you can do to prevent more lung damage is to stop
```

Cell 11: Summary Table

```
# Final summary comparing all three models
summary = pd.DataFrame({
    'Model'          : ['TF-IDF', 'BM25', 'Dense (MiniLM)'],
    'Approach'       : ['Keyword frequency weights', 'Probabilistic ranking', 'Semantic vector search'],
    f'Precision@{K}' : [tfidf_scores[f'Precision@{K}'], bm25_scores[f'Precision@{K}'], dense_scores[f'Precision@
    f'Recall@{K}'    : [tfidf_scores[f'Recall@{K}'],    bm25_scores[f'Recall@{K}'],    dense_scores[f'Recall@{K}
    'MAP'            : [tfidf_scores['MAP'],             bm25_scores['MAP'],             dense_scores['MAP']],
    'MRR'            : [tfidf_scores['MRR'],             bm25_scores['MRR'],             dense_scores['MRR']],
    'Speed'          : ['Very Fast', 'Fast', 'Moderate (GPU recommended for large corpora)'],
    'Handles Semantics': ['No', 'No', 'Yes'],
})

print("\n===== FINAL SUMMARY =====")
print(summary.to_string(index=False))
print("\nConclusion: Dense Retrieval consistently outperforms classical models,")
print("especially on semantically rephrased medical queries.")
```

```
===== FINAL SUMMARY =====
         Model                 Approach  Precision@5  Recall@5    MAP    MRR
        TF-IDF Keyword frequency weights       0.0816     0.408 0.2185 0.2185
          BM25     Probabilistic ranking       0.0908     0.454 0.2569 0.2569
Dense (MiniLM)    Semantic vector search       0.1310     0.655 0.4157 0.4157 Moderate (GPU recommended for large
```

```
Conclusion: Dense Retrieval consistently outperforms classical models,
especially on semantically rephrased medical queries.
```

Conclusion

This notebook implemented and compared three Information Retrieval models on the MedQuAD healthcare dataset:

Model	Best For	Limitation
TF-IDF	Quick baseline, interpretable	Vocabulary mismatch, no semantics
BM25	Better ranking, length-normalized	Still keyword dependent
Dense (MiniLM)	Semantic queries, best accuracy	Needs encoding time upfront

Key finding: Dense retrieval is significantly better at handling the vocabulary gap between how patients phrase questions and how medical documents are written — which is the central challenge in healthcare IR.

```
from google.colab import files
files.download('/content/ir_comparison_chart.png')
```